

Spatial Election Explorer (SEE)

M. August Rossi, Michael Xiao, Emma Ross, Shu Luan

University of Chicago

May 12, 2026

Abstract. We introduce Spatial Election Explorer (SEE), an interactive web application for simulating ranked-choice elections on the Euclidean plane. By encouraging users to iteratively build, experiment, and explore, SEE helps users build visual intuition about the properties and behavior of voting rules.

1 Introduction

Imagine a scenario in which some agents have each ranked a set of alternatives from most preferred to least preferred. How should this set of rankings be consolidated to a single ranking or to a set of winning alternatives that fairly represents the entire group’s preferences? The most familiar setting for this problem is an election, where voters rank candidates to choose their representatives, but the problem setup applies much more broadly: consider a set of friends choosing what movie to watch or restaurant to visit; a committee choosing what candidates to shortlist or hire; or a panel of judges who have ranked competitors in ice skating or ballroom dance.

There are many algorithms for aggregating these group preferences: how should the agents choose which to use? Deliberation among the agents about this decision might rely on the details of the algorithms, or on appeals to axiomatic properties that such algorithms may or may not satisfy. They might say that STV is “more proportional” than bloc plurality for electing committees; that IRV, unlike plurality, “avoids the spoiler effect”; or that plurality exhibits a “center squeeze” effect. However, non-experts may find such arguments to be overly abstract. Methods to visualize the behavior of voting rules can bridge this communication gap and help voters develop intuition about the differences between algorithms.

The core objective of our project is to employ a previously developed technique for visualizing elections as the basis of an interactive election simulator. Allowing users to build, experiment, and iterate upon their own simulations builds intuition in ways that a set of static plots cannot. Combining visualization and iterative experimentation allows users to *see* the properties of the various algorithms and to learn the conditions under which these properties emerge.

Our Methodology. To facilitate visualization, we adopt a two-dimensional spatial model of voters’ pref-

erences. Voters and candidates are jointly embedded in the Euclidean plane, and voters rank candidates in order of increasing distance. Voters thus prefer candidates that are “similar” to themselves. This model is common in both theoretical [1] and empirical [2] research in computational social science. Further, it has natural interpretations: the position of voters and candidates might, for example, represent their positions on two political issues or on two ideological axes.

In the Spatial Election Explorer (SEE), voters and candidates can be placed on the plane by hand or by sampling from the available distributions. Their positions determine a unique preference profile, which can be used as input to any of various voting methods. The winners are displayed, providing the user a visual description how the distribution of winners differs according to the choice of voting rule.

Audience. The intended audience for our application is threefold:

1. Computational social choice (COMSOC) researchers can experiment with various voting rules, testing and refining the intuition they already have for those methods. Visualization may also suggest behavior patterns in these voting rules that can be axiomatized and studied formally.
2. Students studying social choice theory can use the tool to develop intuition for the behavior of voting rules that supplements their learnings in the classroom. More aspirationally, perhaps they might be inspired to pursue research in the area.
3. Policymakers and policy advocates who are considering the implementation of ranked choice voting and wondering what voting rule to use might find value in the tool as they develop intuition for the behavior of and tradeoffs between the rules under consideration.

Related Work. Our work is primarily inspired by Elkind et al. [3], whose experiments generated plots capturing the distribution of winning candidates under various multi-winner voting rules and across varying underlying distributions of voters and candidates. Szufa et al. [4] created visual “maps” of preference profiles by computing swap distances between ballots and applying the multidimensional scaling (MDS) algorithm.

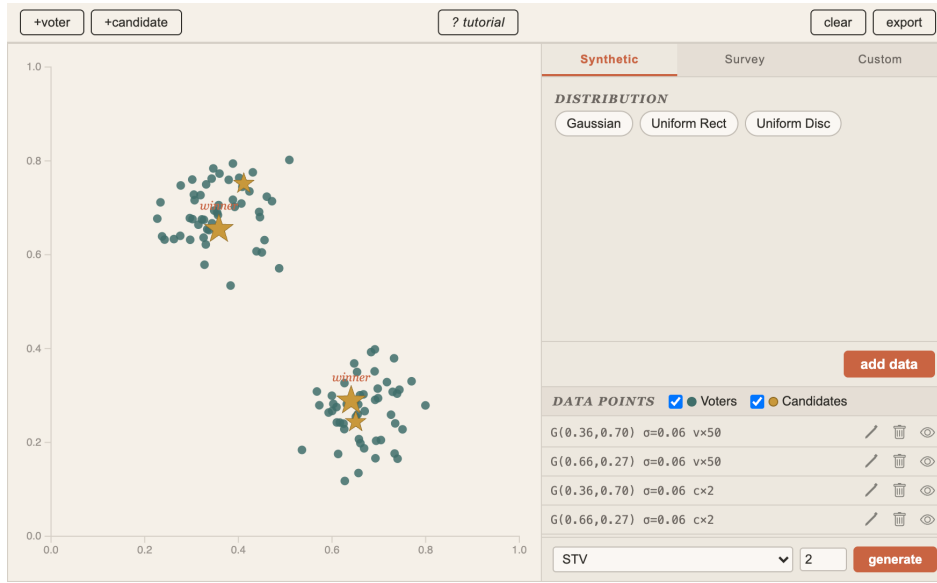


Figure 1: Simulating a multi-winner STV election with the Spatial Election Simulator (SEE).

2 Preliminaries

Spatial Model of Elections. Each voter and candidate has a position in $\mathbb{R}^2$. The pairwise Euclidean distances are computed between all voters and all candidates. A voter’s ballot ranks candidates in order of increasing distance. A collection of such rankings is called a *preference profile*.

Single-Winner Voting Rules. Let there be n voters and m candidates.

Plurality. The candidate with the most first-place votes wins.

Borda. Each candidate’s score is the sum of the number of candidates ranked below them across all ballots. The candidate with the highest score wins.

Instant Runoff Voting (IRV). First-place votes are tallied. The candidate with the fewest votes is eliminated; their supporters’ votes transfer to their next favorite remaining candidate. Repeat until only one candidate remains—they are the winner.

Multi-Winner Voting Rules. Let k be the number of winners, i.e., these rules elect a committee of size k .

Single Non-Transferable Vote (SNTV). First-place votes are tallied. The k candidates with the highest score are elected.

Bloc Plurality. Each candidate’s score is the number of ballots in which they appear in the top k positions. The k highest-scoring candidates are elected.

k -Borda. The k candidates with the highest Borda scores are elected.

STV First-place votes are tallied to give initial candidate scores. Iterate the following process: if any

candidate’s score s is greater than $q = \lfloor \frac{n}{k+1} \rfloor$, elect them, reweight their supporters by a factor of $\frac{s-q}{s}$, and transfer their support to their next-favorite remaining candidate. If no candidate is above the threshold, eliminate the candidate with the lowest score and redistribute their support.

Only one candidate can be elected at a time: if multiple candidates are simultaneously above threshold, the candidate with greater score is elected, and their support is redistributed before any other candidates are elected.

For all voting rules, ties are broken arbitrarily.

3 Design and Implementation

3.1 Tutorial

The tutorial serves to orient the user, introduce the voting rules, and give the user a sense of how to interact with the application.

It begins by defining the spatial model, contextualizing the axes as representing a policy or ideology space, and emphasizing the key assumption: only the distance between voters and candidates matters in determining how voters will fill out their ballots.

Next, it introduces the tool and the primary ways of using it via a series of examples that introduce Plurality, Borda, IRV, and STV. At each step, the tutorial shows an example plot and asks the user to make a prediction of which candidate will win.

The plurality example demonstrates the well-known “spoiler effect”, in which a majority bloc of voters is shut out of representation because they split their votes across

multiple candidates, while the minority bloc is unified in their support of a single candidate.

The Borda example explains how candidates are scored, then demonstrates its tendency to elect centrist “consensus” candidates.

The IRV example shows how runoff voting can avoid the spoiler effect: the plot is set up the same as in the plurality example, but, in this case, one of the candidates supported by the majority bloc wins the election.

Finally, the multi-winner example contrasts bloc plurality with STV. It shows two clusters of voters, one slightly larger than the other, with two candidates per voter cluster. In bloc plurality, the larger cluster can sweep the election, securing both seats on the committee, shutting out the other cluster entirely. However, in STV, each cluster gets one seat on the committee—a more proportional outcome.

3.2 Input

On the plot, voters are blue dots, and candidates are yellow stars. There are four primary methods for placing voters and candidates on the plot.

- **Manual Input.** The user can select to add a candidate or a voter, then click anywhere in the plot to add a point at that location.
- **Synthetic Sampling.** In the sidebar, the user can choose from any of the available distributions. They can set the distribution center by dragging it on the plot, or specifying the desired coordinate position in an input box. They can set distribution parameters with sliders or by entering the desired value in an input box. Then, they can set whether to sample voters or candidates, and set the number of points to be sampled. Finally, clicking “add data” adds these points to the plot.

By stacking sets of samples drawn from different distributions, the user can easily create multiple clusters of voters or candidates.

As in [3], the supported distributions are the Gaussian, the Uniform Rectangle, and the Uniform Disc.

- **Survey Data.** Upon selecting the “Survey” tab in the sidebar, the user can select two “Ideology” axes or political issues to serve as the axes of the plot. Then, they can sample voters from polling data.
- **Data Upload.** If a user has exported a JSON file of the points from a previously-generated plot, they may upload that data.

3.3 Plot Management

Every point manually added and every distribution sampled from appears as a separate line item in the “Data

Points” section of the sidebar. The line describes the parameters that created it. For every distribution line, the following interactions are available:

- **Edit.** Highlights the distribution and reveals its settings; any of the settings (include type, count, position, and parameters) can be changed, and when the user clicks “update”, the points are resampled. The user can click “cancel” at any point in this interaction if they want to keep the old settings.
- **Delete.** Removes this distribution.
- **Hide.** Hides this distribution; it is temporarily disabled for any election runs, but stays listed in the sidebar to be easily re-enabled later.

For any line item representing a manually-placed candidate or voter, only the Delete and Hide options are available.

3.4 Running Elections

At the bottom of the sidebar, the user can choose their desired voting method, select the number of winners, and run the election. Running the election updates the plot, making the winning candidates visually distinct from the others.

3.5 Survey Data

3.6 Implementation

The application front end is built with Svelte and D3 in JS, with Vite as the server. The election algorithms are implemented in Python and compiled to Web Assembly via Pyodide so that they can run in browser.

4 Future Directions

More Voting Rules.

Other Distributions.

Representing Uncertainty.

References

- [1] J. M. Enelow and M. J. Hinich, *The Spatial Theory of Voting* (Cambridge Books 9780521275156), None. Cambridge University Press, Enero-Abr 1984, vol. None.
- [2] N. Boehmer, P. Faliszewski, Ł. Janeczko, A. Kaczmarczyk, G. Lisowski, G. Pierczyński, S. Rey, D. Stolicki, S. Szufa, and T. Wąs, *Guide to numerical experiments on elections in computational social choice*, 2024. arXiv: 2402.11765 [cs.GT].
- [3] E. Elkind, P. Faliszewski, J.-F. Laslier, P. Skowron, A. Slinko, and N. Talmon, *What do multiwinner voting rules do? an experiment over the two-dimensional euclidean domain*, 2019. arXiv: 1901.09217 [cs.GT].
- [4] P. Faliszewski, A. Kaczmarczyk, K. Sornat, S. Szufa, and T. Wąs, “Diversity, agreement, and polarization in elections,” in *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23*, E. Elkind, Ed., Main Track, International Joint Conferences on Artificial Intelligence Organization, Aug. 2023, pp. 2684–2692.